

Phase 2 File Structure

Complete Plugin-Based Directory Structure

```
insiedr/
├── agent/      # Endpoint agent
│   ├── agent.py
│   ├── config.py
│   ├── payload_builder.py
│   ├── transport.py
│   ├── crypto/
│   │   ├── __init__.py
│   │   ├── aesgcm.py
│   │   └── fernet_compat.py
│   ├── collectors/  # Feature collector plugins
│   │   ├── __init__.py
│   │   ├── base.py
│   │   ├── file_scanner.py
│   │   ├── browser_history.py
│   │   ├── usb_monitor.py
│   │   ├── session_monitor.py
│   │   ├── network_monitor.py
│   │   ├── email_monitor.py
│   │   └── process_watcher.py
│   ├── queue/
│   │   ├── __init__.py
│   │   └── local_queue.py  # Offline caching (non-SQLite)
│   └── requirements_agent.txt
├── server/     # Central analysis server
│   ├── app.py
│   ├── config.py
│   └── plugin_registry.py  # Plugin discovery & lifecycle
```

```

├─ crypto/      # Crypto plugin category
│   ├── __init__.py
│   ├── base.py
│   ├── aesgcm_plugin.py
│   ├── fernet_plugin.py
│   └─ plaintext_plugin.py
├─ features/    # Feature extractor plugins
│   ├── __init__.py
│   ├── base.py
│   ├── cert_behavioral.py
│   └─ lanl_edr.py
├─ baseline/    # Baseline engine
│   ├── __init__.py
│   └─ engine.py
├─ detectors/   # Active Phase 2 Detector plugins
│   ├── __init__.py
│   ├── base.py
│   ├── zscore_detector.py
│   ├── random_forest_detector.py # Active Phase 2 Model
│   ├── isolation_forest_detector.py
│   ├── auth_burst_detector.py
│   └─ lstm_forecaster.py      # Active Phase 2 Model
├─ risk/        # Risk aggregation
│   ├── __init__.py
│   └─ aggregator.py
├─ storage/     # Storage plugin category
│   ├── __init__.py
│   ├── base.py
│   ├── postgres_storage.py #PostgreSQL Backend
│   └─ migrations/
│       └─ 001_initial_schema.sql #PostgreSQL schema
├─ api/         # API route blueprints
│   ├── __init__.py
│   ├── ingest.py
│   ├── agents.py
│   └─ logs.py

```

```
| | | |─ anomalies.py
| | | |─ baseline.py
| | | |─ stats.py
| | | |─ health.py
| | |─ dashboard/
| | | |─ templates/
| | | |   |─ dashboard.html
| | |─ requirements_server.txt
|
|─ shared/  # Shared between agent & server
| |─ __init__.py
| |─ protocol.py
| |─ crypto_utils.py
|
|─ tests/
| |─ test_crypto_plugins.py
| |─ test_feature_extractors.py
| |─ test_detectors.py
| |─ test_baseline_engine.py
| |─ test_risk_aggregator.py
| |─ test_storage.py
| |─ test_integration.py
|
|─ scripts/
| |─ generate_keys.py
| |─ migrate_db.py
| |─ simulate_agent.py
|
|─ .env.example
|─ README.md
```

1. Endpoint Agent (**agent/**)

This directory contains the files for the endpoint agent that runs on user machines to collect data.

-

`agent.py` : The main executable script that runs the agent.

-

`config.py` : Stores the configuration settings for the agent.

-

`payload_builder.py` : Packages the collected data into payloads for transmission.

-

`transport.py` : Handles sending the data payloads to the central server.

-

`requirements_agent.txt` : Lists the Python dependencies required to run the agent.

-

`crypto/` : Contains agent-side cryptographic logic, specifically using AES-GCM (`aesgcm.py`) and Fernet (`fernet_compat.py`).

-

`collectors/` : Feature collector plugins that monitor and gather different types of telemetry data. Contains a base template (`base.py`) and specific monitors for files, browser history, USBs, sessions, networks, emails, and processes.

-

`queue/` : Handles local data queueing. Includes `local_queue.py` for offline caching without using SQLite.

2. Central Analysis Server (`server/`)

This directory contains the central server that receives, stores, and analyzes the data from the agents.

-

`app.py` : The main application script that runs the server.

-

`config.py` : Stores the configuration settings for the server.

-

`plugin_registry.py` : Manages the discovery and lifecycle of various server plugins.

-

`requirements_server.txt` : Lists the Python dependencies needed for the server.

-

`crypto/` : Crypto plugin category for decrypting agent payloads, including AES-GCM, Fernet, and plaintext options.

-

`features/` : Feature extractor plugins (like `cert_behavioral.py` and `lanl_edr.py`) that pull relevant data points from the ingested telemetry.

-

`baseline/` : The baseline engine (`engine.py`) that establishes normal behavior patterns.

-

`detectors/` : Active Phase 2 detector plugins used to find anomalies and threats. This includes Z-Score, Random Forest, Isolation Forest, Auth Burst, and an LSTM Forecaster.

-

`risk/` : Contains `aggregator.py` which handles risk aggregation from the different detectors.

-

`storage/` : Storage plugins handling database operations. It relies exclusively on a PostgreSQL backend (`postgres_storage.py`).

-

`migrations/` : Contains database schemas, such as the initial PostgreSQL schema (`001_initial_schema.sql`).

-

`api/` : Contains the API route blueprints for ingesting data, managing agents, logs, anomalies, baselines, stats, and health checks.

-

`dashboard/` : Contains UI templates, specifically `dashboard.html` , to visualize the server data.

3. Shared Resources (`shared/`)

Files shared between both the agent and the server.

-

`protocol.py` : Defines the communication rules and formatting between the agent and server.

-

`crypto_utils.py` : Shared cryptographic utility functions.

4. Tests (`tests/`)

Contains test scripts to ensure all parts of the system work correctly.

- Includes isolated test scripts for crypto plugins, feature extractors, detectors, the baseline engine, risk aggregator, storage, and overall integration.

5. Utility Scripts (`scripts/`)

Helpful scripts for setting up and managing the environment.

-

`generate_keys.py` : Generates cryptographic keys for secure communication.

-

`migrate_db.py` : Applies database migrations/updates.

-

`simulate_agent.py` : Simulates an endpoint agent for testing the server without needing a real deployment.

6. Root Level Files

-

`.env.example` : A template file for setting environment variables.

-

`README.md` : The main documentation file for the project.

(Note: The `__init__.py` files scattered throughout simply tell Python to treat those directories as packages).

Would you like me to elaborate on the specific role of any particular plugin or directory listed above?